

Futcamedic Database Schema

PostgreSQL Schema Documentation

Engineering Team
Futcamedic

May 5, 2026

Futcamedic Database Schema

Database: PostgreSQL (hosted on Supabase) Row Level Security (RLS): Enabled on all tables
Multi-Tenancy: All tables have `school_id` column

Entity Relationship Summary

```
schools (1) -----> (N) users
schools (1) -----> (N) categories
schools (1) -----> (N) students
schools (1) -----> (N) attendances
schools (1) -----> (N) payments
schools (1) -----> (N) events
schools (1) -----> (N) trainings
schools (1) -----> (N) venues
schools (1) -----> (N) profile_information
schools (1) -----> (N) deleted_students
schools (1) -----> (N) teacher_permissions

categories (1) -----> (N) students
categories (1) -----> (N) category_teachers
categories (1) -----> (N) attendances
categories (1) -----> (N) events
categories (1) -----> (N) trainings

users (1) -----> (N) students (as parent)
users (1) -----> (N) category_teachers (as teacher)
users (1) -----> (N) attendances (as teacher)
users (1) -----> (N) payments (as student or teacher)
users (1) -----> (N) teacher_permissions

events (1) -----> (N) trainings

payments (1) -----> (N) payment_students
students (1) -----> (N) payment_students
```

Table Definitions

1. schools

Tenant principal — each football academy is a school.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique school identifier
name	VARCHAR(255)	NOT NULL	Academy name
created_at	TIME- STAMP WITH TIME ZONE	DEFAULT NOW()	Record creation timestamp

RLS Policy: Users can only see their own school (`id = (SELECT school_id FROM users WHERE id = auth.uid())`)

2. users

Links to Supabase `auth.users` — stores role and school association.

Column	Type	Constraints	Description
id	UUID	PK, FK → <code>auth.users(id)</code> ON DELETE CASCADE	Same as Supabase Auth user ID
school_id	UUID	FK → <code>schools(id)</code> ON DELETE CASCADE, NOT NULL	Tenant association
role	VAR- CHAR(50)	CHECK IN (<code>'super_admin'</code> , <code>'admin'</code> , <code>'pro- fesor'</code> , <code>'padre'</code> , <code>'alumno'</code>)	User role
full_name	VARCHAR(255)	NOT NULL	User's full name
created_at	TIME- STAMP WITH TIME ZONE	DEFAULT NOW()	Record creation timestamp

RLS Policy: Users can only see users from same school. **Note:** `auth.users` is managed by Supabase — email, password, email confirmation handled there.

3. categories

Football categories based on birth year.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique category ID
school_id	UUID	FK → schools(id) ON DELETE CASCADE, NOT NULL	Tenant association
birth_year	INT	NOT NULL	Birth year for age grouping
name	VARCHAR(100)	NOT NULL	Category name (e.g., "U-12")
created_at	TIME- STAMP WITH TIME ZONE	DEFAULT NOW()	Record creation timestamp

Unique Constraint: `UNIQUE(school_id, birth_year)` — prevents duplicate categories per school per year. **RLS Policy:** Users can only see categories from same school.

4. category_teachers

Many-to-many relationship between teachers and categories.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique record ID
school_id	UUID	FK → schools(id) ON DELETE CASCADE, NOT NULL	Tenant association
category_id	UUID	FK → categories(id) ON DELETE CASCADE, NOT NULL	Linked category
teacher_id	UUID	FK → users(id) ON DELETE CASCADE, NOT NULL	Linked teacher
created_at	TIME- STAMP WITH TIME ZONE	DEFAULT NOW()	Record creation timestamp

Unique Constraint: `UNIQUE(category_id, teacher_id)` — prevents duplicate assignments. **RLS Policy:** Users can only see assignments from same school.

5. students

Football academy students.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique student ID
school_id	UUID	FK → schools(id) ON DELETE CASCADE, NOT NULL	Tenant association
category_id	UUID	FK → categories(id) ON DELETE CASCADE, NOT NULL	Assigned category
parent_id	UUID	FK → users(id) ON DELETE SET NULL	Linked parent (padre role)
full_name	VARCHAR(255)	NOT NULL	Student's full name
birth_date	DATE	NOT NULL	Student's birth date
uniform_delivered	BOOLEAN	DEFAULT FALSE	Whether uniform was delivered
status	VAR- CHAR(20)	CHECK IN ('ac- tivo','beca','pendiente','in- activo'), DEFAULT 'activo'	Enrollment status
created_at	TIME- STAMP WITH TIME ZONE	DEFAULT NOW()	Record creation timestamp

RLS Policy: Users can only see students from same school. Parents only see their own children (`parent_id = auth.uid()`). **Status Values:** - `activo` — Active, paying student - `beca` — Scholarship student (no payment required) - `pendiente` — Pending enrollment - `inactivo` — Inactive, no longer attending

6. attendances

Attendance records for training sessions and matches.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique attendance ID
school_id	UUID	FK → schools(id) ON DELETE CASCADE, NOT NULL	Tenant association
student_id	UUID	FK → students(id) ON DELETE CASCADE, NOT NULL	Student being marked
category_id	UUID	FK → categories(id) ON DELETE CASCADE, NOT NULL	Category context
teacher_id	UUID	FK → users(id) ON DELETE SET NULL	Teacher who took attendance
training_id	UUID	FK → trainings(id) ON DELETE SET NULL	Linked training session
date	DATE	NOT NULL	Attendance date
type	VAR- CHAR(50)	CHECK IN ('entre- namiento','partido')	Session type
present	BOOLEAN	NOT NULL, DEFAULT FALSE	Whether student was present
created_at	TIME- STAMP WITH TIME ZONE	DEFAULT NOW()	Record creation timestamp

Unique Constraint: `UNIQUE(student_id, date, type)` — prevents duplicate attendance for same student/date/type. **RLS Policy:** Users can only see attendances from same school.

7. payments

Payment records for student monthly fees and teacher payroll.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique payment ID
school_id	UUID	FK → schools(id) ON DELETE CASCADE, NOT NULL	Tenant association
amount	NUMERIC(10,2)	NOT NULL	Payment amount
payment_date	DATE	NOT NULL	Date payment was made
payment_type	VAR- CHAR(50)	CHECK IN ('mensuali- dad','pago_profesor')	Payment type
student_id	UUID	FK → students(id) ON DELETE SET NULL	Linked student (for mensu- alidad)
teacher_id	UUID	FK → users(id) ON DELETE SET NULL	Linked teacher (for pago_profesor)
description	TEXT		Payment description/con- cept
created_at	TIME- STAMP WITH TIME ZONE	DEFAULT NOW()	Record creation timestamp

Payment Types: - mensualidad — Student monthly fee - pago_profesor — Teacher payroll payment

RLS Policy: Users can only see payments from same school.

8. payment_students

Links multiple students to a single group payment.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique record ID
payment_id	UUID	FK → payments(id) ON DELETE CASCADE, NOT NULL	Parent payment record
student_id	UUID	FK → students(id) ON DELETE CASCADE, NOT NULL	Linked student
amount	NUMERIC(10,2)	NOT NULL	Amount allocated to this student
created_at	TIME- STAMP WITH TIME ZONE	DEFAULT NOW()	Record creation timestamp

Unique Constraint: UNIQUE(payment_id, student_id) — prevents duplicate student entries in same payment. **RLS Policy:** Inherits from payments table policy.

9. venues

Training venues / fields.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique venue ID
school_id	UUID	FK → schools(id) ON DELETE CASCADE, NOT NULL	Tenant association
name	VARCHAR(255)	NOT NULL	Venue name
address	TEXT		Physical address
notes	TEXT		Additional notes
created_at	TIME- STAMP WITH TIME ZONE	DEFAULT NOW()	Record creation timestamp

RLS Policy: Users can only see venues from same school.

10. events

Master events with recurrence rules.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique event ID
school_id	UUID	FK → schools(id) ON DELETE CASCADE, NOT NULL	Tenant association
category_id	UUID	FK → categories(id) ON DELETE CASCADE, NOT NULL	Linked category
venue_id	UUID	FK → venues(id) ON DELETE SET NULL	Linked venue
date	DATE	NOT NULL	Event start date
start_time	TIME		Event start time
type	VAR- CHAR(50)	CHECK IN ('entre- namiento','partido')	Event type
description	TEXT		Event description
is_recurring	BOOLEAN	DEFAULT FALSE	Whether event recurs
recurring_weeks	INT		Number of weeks for recur- rence
recurring_end_date	DATE		End date for recurrence
recurrence_rule	JSONB		Recurrence configuration (frequency, days)
created_at	TIME- STAMP WITH TIME ZONE	DEFAULT NOW()	Record creation timestamp

Recurrence Rule JSONB Example (Weekly):

```
{
  "frequency": "weekly",
  "days": ["Monday", "Wednesday", "Friday"]
}
```

Recurrence Rule JSONB Example (Biweekly):

```
{
  "frequency": "biweekly",
  "day": "Saturday"
}
```

RLS Policy: Users can only see events from same school.

11. trainings

Individual training sessions generated from recurring events.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique training ID
school_id	UUID	FK → schools(id) ON DELETE CASCADE, NOT NULL	Tenant association
event_id	UUID	FK → events(id) ON DELETE CASCADE, NOT NULL	Parent recurring event
category_id	UUID	FK → categories(id) ON DELETE CASCADE, NOT NULL	Linked category
venue_id	UUID	FK → venues(id) ON DELETE SET NULL	Linked venue
date	DATE	NOT NULL	Training date
start_time	TIME		Training start time
type	VAR- CHAR(50)	CHECK IN ('entre- namiento','partido')	Session type
is_completed	BOOLEAN	DEFAULT FALSE	Whether attendance was taken
is_cancelled	BOOLEAN	DEFAULT FALSE	Whether session was can- celled
created_at	TIME- STAMP WITH TIME ZONE	DEFAULT NOW()	Record creation timestamp

RLS Policy: Users can only see trainings from same school.

12. profile_information

Additional personal details for users.

Column	Type	Constraints	Description
id	UUID	PK, FK → users(id) ON DELETE CASCADE	Same as users.id
school_id	UUID	FK → schools(id) ON DELETE CASCADE, NOT NULL	Tenant association
phone	VAR-CHAR(20)		Phone number
address	TEXT		Physical address
birth_date	DATE		Birth date
gender	VAR-CHAR(20)		Gender
emergency_contact_name	VAR-CHAR(255)		Emergency contact name
emergency_contact_phone	VAR-CHAR(20)		Emergency contact phone
medical_notes	TEXT		Medical notes/allergies
updated_at	TIME-STAMP WITH TIME ZONE	DEFAULT NOW()	Last update timestamp

RLS Policy: Users can only see profiles from same school.

13. deleted_students

Audit trail for deleted students.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique record ID
school_id	UUID	FK → schools(id) ON DELETE CASCADE, NOT NULL	Tenant association
original_student_id	UUID	NOT NULL	Original student ID (before deletion)
full_name	VARCHAR(255)	NOT NULL	Student name at time of deletion
birth_date	DATE	NOT NULL	Student birth date
category_id	UUID		Category at time of deletion
parent_id	UUID		Parent at time of deletion
deleted_at	TIME- STAMP WITH TIME ZONE	DEFAULT NOW()	Deletion timestamp
deleted_by	UUID	FK → users(id) ON DELETE SET NULL	User who deleted the stu- dent

RLS Policy: Only admin/super_admin from same school can view.

14. teacher_permissions

Granular permissions for teachers.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique record ID
school_id	UUID	FK → schools(id) ON DELETE CASCADE, NOT NULL	Tenant association
teacher_id	UUID	FK → users(id) ON DELETE CASCADE, NOT NULL	Linked teacher
can_manage_students	BOOLEAN	DEFAULT TRUE	Can CRUD students
can_manage_events	BOOLEAN	DEFAULT TRUE	Can manage events
can_view_finances	BOOLEAN	DEFAULT FALSE	Can view payment info
can_manage_payments	BOOLEAN	DEFAULT FALSE	Can record payments
can_take_attendance	BOOLEAN	DEFAULT TRUE	Can take attendance
can_manage_categories	BOOLEAN	DEFAULT FALSE	Can manage categories
created_at	TIME- STAMP WITH TIME ZONE	DEFAULT NOW()	Record creation timestamp

Unique Constraint: UNIQUE(teacher_id) — one permission record per teacher. **RLS Policy:** Only admin/super_admin from same school can view/manage.

Row Level Security (RLS) Policies

All tables have RLS enabled. The standard tenant isolation policy:

```
CREATE POLICY "Aislamiento tenant <table>" ON public.<table>
FOR ALL USING (school_id = (SELECT school_id FROM public.users WHERE id =
auth.uid()));
```

Exception: super_admin role bypasses RLS via service role key in backend.

Indexes (Recommended)

For performance, consider adding these indexes:

```
-- Frequently queried columns
CREATE INDEX idx_users_school_id ON users(school_id);
CREATE INDEX idx_students_school_id ON students(school_id);
CREATE INDEX idx_students_category_id ON students(category_id);
CREATE INDEX idx_students_parent_id ON students(parent_id);
CREATE INDEX idx_attendances_school_id ON attendances(school_id);
CREATE INDEX idx_attendances_student_id ON attendances(student_id);
CREATE INDEX idx_attendances_date ON attendances(date);
CREATE INDEX idx_payments_school_id ON payments(school_id);
CREATE INDEX idx_payments_student_id ON payments(student_id);
CREATE INDEX idx_payments_payment_date ON payments(payment_date);
```

```
CREATE INDEX idx_events_school_id ON events(school_id);
CREATE INDEX idx_events_category_id ON events(category_id);
CREATE INDEX idx_trainings_event_id ON trainings(event_id);
CREATE INDEX idx_trainings_date ON trainings(date);
```